

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

RASHMITA R KHARVI (1BM25CS502)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Rashmita R Kharvi (1BM25CS502), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil

Assistant Professor

Department of CSE

BMSCE, Bengaluru

Dr. Kavitha Sooda

Professor and Head

Department of CSE

BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. a) FCFS b) SJF (Preemptive and Non-Preemptive) c) Priority (Preemptive and Non-Preemptive) d) Round Robin (Experiment with different quantum sizes for RR algorithm)	1-37
2.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	38-45
3.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	46-62
4.	Write a C program to simulate: a) Producer-Consumer problem using semaphores. b) Dining-Philosopher's problem	63-73
5.	Write a C program to simulate: a) Bankers' algorithm for the purpose of deadlock avoidance. b) Deadlock Detection	74-91
6.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	92-100
7.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	101-110

Course Outcomes:

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

1. Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS

b) SJF

c) Priority

d) Round Robin (Experiment with different quantum sizes for RR algorithm)

a) FCFS

Program :

```
#include<stdio.h>

int main() {

    int n, pid[20], at[20], bt[20], ct[20], tat[20], wt[20], i;

    printf("Name: Rashmita Kharvi\n");

    printf("USN : 1BM25CS502\n");

    printf("Enter number of processes: ");

    scanf("%d",&n);

    for(i=0;i<n;i++) {

        printf("\nEnter Process ID: ");

        scanf("%d",&pid[i]);

        printf("Enter Arrival Time: ");

        scanf("%d",&at[i]);

        printf("Enter Burst Time: ");

        scanf("%d",&bt[i]);

    }

    ct[0]=at[0]+bt[0];
```

```

for(i=1;i<n;i++) {

    if(ct[i-1]<at[i])

        ct[i]=at[i]+bt[i];

    else

        ct[i]=ct[i-1]+bt[i];

}

for(i=0;i<n;i++) {

    tat[i]=ct[i]-at[i];

    wt[i]=tat[i]-bt[i];

}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)

    printf("%d\t%d\t%d\t%d\t%d\t%d\n",

        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);

return 0;

}

```

Output

```

Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of processes: 3

Enter Process ID: 1
Enter Arrival Time: 4
Enter Burst Time: 6

Enter Process ID: 2
Enter Arrival Time: 5
Enter Burst Time: 6

Enter Process ID: 3
Enter Arrival Time: 5
Enter Burst Time: 6

```

PID	AT	BT	CT	TAT	WT
1	4	6	10	6	0
2	5	6	16	11	5
3	5	6	22	17	11

OBSERVATION COPY

LAB - PROGRAM - 1

Page No. _____
Date: _____

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time

a) FCFS

```
#include <stdio.h>
int main() {
    int n, i;
    int at[10], bt[10], ct[10], tat[10], wt[10];
    printf("Enter number of processes:");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter AT and BT of P%d:", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }
    ct[0] = at[0] + bt[0];
    for (i = 1; i < n; i++) {
        if (ct[i-1] < at[i])
            ct[i] = at[i] + bt[i];
        else
            ct[i] = ct[i-1] + bt[i];
    }
    for (i = 0; i < n; i++) {
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
    printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
    for (i = 0; i < n; i++)
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
}
```

```
tot[i], wt[i]);
```

```
return 0;
```

```
}
```

OUTPUT:

Enter Number of processes : 4

Enter AT and BT of P1 : 0 2

Enter AT and BT of P2 : 3 4

Enter AT and BT of P3 : 5 2

Enter AT and BT of P4 : 1 7

P	AT	BT	CT	TAT	WT
P ₁	0	2	2	2	0
P ₂	3	4	7	4	0
P ₃	5	2	9	4	2
P ₄	1	7	16	15	8

[Handwritten signature/initials over the table]

b)

i. **SJF (Non-Preemptive)**

Program :

```
#include<stdio.h>

int main() {

    int n,pid[20],at[20],bt[20],ct[20],tat[20],wt[20];

    int done[20]={0},i,j,time=0,completed=0,min,pos;

    printf("Name: Rashmita Kharvi\n");

    printf("USN : 1BM25CS502\n");

    printf("Enter number of processes: ");

    scanf("%d",&n);

    for(i=0;i<n;i++) {

        printf("\nEnter Process ID: ");

        scanf("%d",&pid[i])

        printf("Enter Arrival Time: ");

        scanf("%d",&at[i]);

        printf("Enter Burst Time: ");

        scanf("%d",&bt[i]);

    }

    while(completed<n) {

        min=9999;

        pos=-1;

        for(i=0;i<n;i++) {

            if(at[i]<=time && !done[i] && bt[i]<min) {
```

```

        min=bt[i];

        pos=i;
    }
}

if(pos==-1){

    time++;

    continue;

}

time+=bt[pos];

ct[pos]=time;

tat[pos]=ct[pos]-at[pos];

wt[pos]=tat[pos]-bt[pos];

done[pos]=1;

completed++;

}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)

    printf("%d\t%d\t%d\t%d\t%d\t%d\n",

        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);

return 0;

}

```

Output

```
Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of processes: 3
```

```
Enter Process ID: 1
Enter Arrival Time: 3
Enter Burst Time: 2
```

```
Enter Process ID: 2
Enter Arrival Time: 4
Enter Burst Time: 4
```

```
Enter Process ID: 3
Enter Arrival Time: 4
Enter Burst Time: 6
```

PID	AT	BT	CT	TAT	WT
1	3	2	5	2	0
2	4	4	9	5	1
3	4	6	15	11	5

OBSERVATION COPY

Page No. _____

Date: _____

b) SJF Non-Preemptive

```
#include <stdio.h>
int main() {
    int n, i, time = 0, completed = 0, smallest;
    int at[10], bt[10], ct[10], tat[10], wt[10];
    done[10] = {0};
    printf("Enter number of processes:");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter AT and BT of P%d:", i+1);
        scanf("%d %d", &at[i], &bt[i]);
    }
    while (completed < n) {
        smallest = -1;
        for (i = 0; i < n; i++) {
            if (at[i] <= time && done[i] == 0) {
                if (smallest == -1 || bt[i] < bt[smallest])
                    smallest = i;
            }
        }
        if (smallest == -1)
            time++;
        else {
            time += bt[smallest];
            ct[smallest] = time;
            tat[smallest] = ct[smallest] - at[smallest];
            wt[smallest] = tat[smallest] - bt[smallest];
            done[smallest] = 1;
            completed++;
        }
    }
}
```

```

    }
    printf("\nP1\tAT\tBT\tCT\tTAT\tWT\n");
    for (i=0; i<n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1,
            at[i], bt[i], ct[i], tat[i], wt[i]);
    }
    return 0;
}

```

OUTPUT:

Enter number of processes: 4

Enter AT and BT of P1: 1 4

Enter AT and BT of P2: 2 5

Enter AT and BT of P3: 7 2

Enter AT and BT of P4: 8 9

P	AT	BT	CT	TAT	WT
P ₁	1	4	5	4	0
P ₂	2	5	10	8	3
P ₃	7	2	12	5	3
P ₄	8	9	21	13	4

ii. SJF(Preemptive)

Program :

```
#include<stdio.h>

int main() {

    int n, pid[20], at[20], bt[20], rt[20];

    int ct[20], tat[20], wt[20];

    int i, time=0, done=0, min;

    printf("Name: Rashmita Kharvi\n");

    printf("USN : 1BM25CS502\n");

    printf("Enter number of processes: ");

    scanf("%d",&n);

    for(i=0;i<n;i++) {

        printf("\nEnter Process ID: ");

        scanf("%d",&pid[i]);

        printf("Enter Arrival Time: ");

        scanf("%d",&at[i]);

        printf("Enter Burst Time: ");

        scanf("%d",&bt[i]);

        rt[i]=bt[i];

    }

    while(done<n) {

        min=-1;

        for(i=0;i<n;i++) {

            if(at[i]<=time && rt[i]>0 &&
```

```

        (min==-1 || rt[i]<rt[min]))

        min=i;
    }

    if(min==-1) {

        time++;

        continue;

    }

    rt[min]--;

    time++;

    if(rt[min]==0) {

        done++;

        ct[min]=time;

        tat[min]=ct[min]-at[min];

        wt[min]=tat[min]-bt[min];

    }

}

printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for(i=0;i<n;i++)

    printf("%d\t%d\t%d\t%d\t%d\t%d\n",

        pid[i],at[i],bt[i],ct[i],tat[i],wt[i]);

return 0;

}

```

OUTPUT

```
Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of processes: 3

Enter Process ID: 1
Enter Arrival Time: 0
Enter Burst Time: 5

Enter Process ID: 2
Enter Arrival Time: 2
Enter Burst Time: 4

Enter Process ID: 3
Enter Arrival Time: 5
Enter Burst Time: 6
```

PID	AT	BT	CT	TAT	WT
1	0	5	5	5	0
2	2	4	9	7	3
3	5	6	15	10	4

OBSERVATION COPY

Page No. _____

Date: _____

c) Priority SJF Preemptive.

```
#include <stdio.h>
int main() {
    int n, i, time = 0, completed = 0, smallest;
    int at[10], bt[10], rt[10], ct[10], tot[10],
    wt[10];
    printf("Enter number of process: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter AT and BT of P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }
    while (completed < n) {
        smallest = -1;
        for (i = 0; i < n; i++) {
            if (at[i] <= time && rt[i] > 0) {
                if (smallest == -1 || rt[i] < rt[smallest])
                    smallest = i;
            }
        }
        if (smallest == -1)
            time++;
        else {
            at[smallest]--;
            time++;
            if (at[smallest] == 0) {
                ct[smallest] = time;
                tot[smallest] = ct[smallest] +
                    at[smallest];
            }
        }
    }
}
```

```

    wt[smallest] = tat[smallest] - bt
    [smallest];
    completed++;
}
}

```

```

printf("\nP\tAT\tBT\tCT\tTAT\tWT\n");
for (i=0; i<n; i++)
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i], wt[i]);
return 0;
}

```

OUTPUT:

Enter number of processes: 4

Enter AT and BT of P1: 2 1

Enter AT and BT of P2: 4 2

Enter AT and BT of P3: 5 3

Enter AT and BT of P4: 6 4

P	AT	BT	CT	TAT	WT
P ₁	2	1	3	1	0
P ₂	4	2	6	2	0
P ₃	5	3	9	4	1
P ₄	6	4	13	7	3

c) Priority Scheduling(Non-Preemptive)

Program :

```
#include<stdio.h>

int main() {

    int n,pid[20],at[20],bt[20],pr[20],ct[20],tat[20],wt[20],rt[20];

    int done[20]={0},time=0,completed=0,i,pos,min;

    float avgwt=0,avgtat=0;

    printf("Name: Rashmita Kharvi\n");

    printf("USN : 1BM25CS502\n")

    printf("Enter number of processes: ");

    scanf("%d",&n);

    printf("\nEnter Process ID:\n");

    for(i=0;i<n;i++) scanf("%d",&pid[i]);

    printf("Enter Arrival Time:\n");

    for(i=0;i<n;i++) scanf("%d",&at[i]);

    printf("Enter Burst Time:\n");

    for(i=0;i<n;i++) scanf("%d",&bt[i]);

    printf("Enter Priority (Lower number = Higher priority):\n");

    for(i=0;i<n;i++) scanf("%d",&pr[i]);

    while(completed<n){

        pos=-1;

        min=999;

        for(i=0;i<n;i++)

            if(!done[i] && at[i]<=time && pr[i]<min){
```

```

        min=pr[i];

        pos=i;
    }
    if(pos==-1){

        time++;

        continue;
    }

    rt[pos]=time-at[pos];

    time+=bt[pos];

    ct[pos]=time;

    tat[pos]=ct[pos]-at[pos];

    wt[pos]=tat[pos]-bt[pos];

    avgwt+=wt[pos];

    avgtat+=tat[pos];

    done[pos]=1;

    completed++;
}

printf("\nPriority Scheduling (Non-Preemptive)\n\n");

printf("PID\tPR\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(i=0;i<n;i++)

    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",

        pid[i],pr[i],at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);

printf("\nAverage Turnaround Time = %.2f",avgtat/n);

printf("\nAverage Waiting Time = %.2f",avgwt/n);

```

```
    return 0;

}
```

Output

```
Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of processes: 5

Enter Process ID:
1 2 3 4 5
Enter Arrival Time:
0 3 5 6 9
Enter Burst Time:
4 5 3 6 2
Enter Priority (Lower number = Higher priority):
3 4 2 1 5

Priority Scheduling (Non-Preemptive)

PID    PR    AT    BT    CT    TAT    WT    RT
1       3     0     4     4     4     0     0
2       4     3     5     9     6     1     1
3       2     5     3    18    13    10    10
4       1     6     6    15     9     3     3
5       5     9     2    20    11     9     9

Average Turnaround Time = 8.60
Average Waiting Time = 4.60
```

OBSERVATION COPY

Page No. _____

Date: _____

Priority Scheduling non-pre-emptive.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i, current_time = 0, completed_count = 0;
```

```
    printf("Enter number of processes:");  
    scanf("%d", &n);
```

```
    int pid[n], at[n], bt[n], pr[n];  
    int ct[n], tat[n], wt[n], completed[n];  
    float avg_wt = 0; avg_tat = 0;
```

```
    for (i = 0; i < n; i++) {
```

```
        printf("InProcess %d ID:", i+1);  
        scanf("%d", &pid[i]);
```

```
        printf("Arrival Time:");  
        scanf("%d", &at[i]);
```

```
        printf("Burst Time:");  
        scanf("%d", &bt[i]);
```

```
        printf("Priority:");  
        scanf("%d", &pr[i]);
```

```
        completed[i] = 0;
```

```
    }
```

```
    while (completed_count < n) {
```

```
        int highest_priority = 9999;
```

```
        int selected = -1;
```

```

for (i = 0; i < n; i++) {
    if (at[i] <= current_time &&
        completed[i] == 0) {
        if (pri[i] < highest_priority) {
            highest_priority = pri[i];
            selected = i;
        }
    }
}

```

```

if (selected == -1) {
    current_time++;
}
else {

```

```

    int start_time = current_time;

```

```

    ct[selected] = start_time +

```

```

        bt[selected];

```

```

    tat[selected] = ct[selected] -
        at[selected];

```

```

    wt[selected] = tat[selected] -
        bt[selected];

```

```

    avg_tat += tat[selected];

```

```

    avg_wt += wt[selected];

```

```

    current_time = ct[selected];

```

```

    completed[selected] = 1;

```

```

    completed_count++;
}

```

```

}

```

```

avg_tat /= n;

```

```

avg_wt /= n;

```

```
printf("\n PID\tAT\tBT\tPR\tWT\t
\tTAT\n");
```

```
for(i=0; i<n; i++) {
```

```
printf("%d\t%d\t%d\t%d\t
\t%d\t%d\n",
```

```
pid[i], at[i], bt[i], pr[i], wt[i],
tat[i]);
```

```
}
```

```
printf("\n Average Waiting Time = %.2f",
avg-wt);
```

```
printf("\n Average Waiting Turn Around
Time = %.2f\n", avg-tat);
```

```
return 0;
```

```
}
```

Output:

Enter Number of Processes : 5

Enter Process ID : 1

Enter Arrival Time : 0

Enter Burst Time : 3

Enter Priority : 5

Enter Process ID : 2

Enter Arrival Time : 2

Enter Burst Time : 2

Enter Priority : 3

Enter Process ID : 3

Enter Arrival Time : 3

Enter Burst Time : 5

Enter Priority : 2

Enter Process ID : 4

Enter Arrival Time : 4

Enter Burst Time : 4

Enter Priority : 4

Enter Process ID : 5

Enter Arrival Time : 6

Enter Burst Time : 1

Enter Priority : 1

PID	AT	BT	PR	CT	WT	TAT	RT
1	0	3	5	3	0	3	0
2	2	2	3	11	7	9	7
3	3	5	2	8	0	5	0
4	4	4	4	15	7	11	7
5	6	1	1	9	2	3	2

Priority Scheduling(Preemptive)

Program:

```
#include <stdio.h>

int main()
{
    int n, i;

    int pid[10], at[10], bt[10], pr[10], rt[10];

    int ct[10], tat[10], wt[10];

    int completed = 0, current_time = 0, min;

    float avg_tat = 0, avg_wt = 0;

    printf("\n\tNAME : R NANDINI\tUSN : 1BM25CS497\tSECTION : 'O'\n\n");

    printf("Enter number of processes: ");

    scanf("%d", &n);

    printf("Enter Arrival Time:\n");

    for(i = 0; i < n; i++)
    {
        pid[i] = i + 1;

        scanf("%d", &at[i]);
    }

    printf("Enter Burst Time:\n");

    for(i = 0; i < n; i++)
    {
        scanf("%d", &bt[i]);

        rt[i] = bt[i];
    }
}
```

```

}

printf("Enter Priority:\n");

for(i = 0; i < n; i++)

    scanf("%d", &pr[i]);

while(completed < n)

{

    min = -1;

    for(i = 0; i < n; i++)

    {

        if(at[i] <= current_time && rt[i] > 0)

        {

            if(min == -1 || pr[i] < pr[min])

                min = i;

        }

    }

    if(min == -1)

    {

        current_time++;

    }

    else

    {

        rt[min]--;

        current_time++;

        if(rt[min] == 0)

```

```

    {
        ct[min] = current_time;

        tat[min] = ct[min] - at[min];

        wt[min] = tat[min] - bt[min];

        completed++;

        avg_tat += tat[min];

        avg_wt += wt[min];
    }
}

printf("\nPriority Scheduling (Preemptive)\n");

printf("PID\tAT\tBT\tPR\tCT\tTAT\tWT\n");

for(i = 0; i < n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",

        pid[i], at[i], bt[i], pr[i],

        ct[i], tat[i], wt[i]);
}

printf("\nAverage Turnaround Time = %.2f", avg_tat / n);

printf("\nAverage Waiting Time = %.2f\n", avg_wt / n);

return 0;
}

```

OUTPUT:

```
Enter number of processes: 5
Enter Arrival Time:
0 2 3 4 6
Enter Burst Time:
3 2 5 4 1
Enter Priority:
5 3 2 4 1

Priority Scheduling (Higher number = Higher priority)
PID    AT    BT    PR    CT    TAT    WT
P1      0     3     5     3     3     0
P2      2     2     3     5     3     1
P3      3     5     2    14    11     6
P4      4     4     4     9     5     1
P5      6     1     1    15     9     8

Average Turnaround Time = 6.20
Average Waiting Time = 3.20

Process returned 0 (0x0)   execution time : 94.714 s
Press any key to continue.
```

Priority Scheduling Preemptive

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i, time = 0, completed = 0, choice;
```

```
    int at[20], bt[20], pr[20], rt[20];
```

```
    int ct[20], tat[20], ctt[20];
```

```
    float avg-tat = 0; avg-ctt = 0;
```

```
    printf("Enter number of processes:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter priorities (1 for higher  
number, higher priority, 2 for lower number  
means, lower priority):");
```

```
    scanf("%d", &choice);
```

```
    printf("Enter priorities:\n");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &pr[i]);
```

```
    printf("Enter arrival times:\n");
```

```
    for (i = 0; i < n; i++)
```

```
        scanf("%d", &at[i]);
```

```
    printf("Enter burst times:\n");
```

```
    for (i = 0; i < n; i++) {
```

```
        scanf("%d", &bt[i]);
```

```
        at[i] = bt[i];
```

```
    }
```

```
    while (completed < n) {
```

```
        int selected = -1;
```

```

for (i = 0; i < n; i++) {
    if (at[i] <= time && rt[i] > 0) {
        if (selected == -1)
            selected = i;
        else {
            if (choice == 1) {
                if (pr[i] > pr[selected])
                    selected = i;
            }
            else {
                if (pn[i] < pn[selected])
                    selected = i;
            }
        }
    }
}

if (selected == -1) {
    time++;
}
else {
    at[selected]--;
    time++;

    if (at[selected] == 0) {
        ct[selected] = time;
        tat[selected] = ct[selected] -
            at[selected];
        wct[selected] = tat[selected] -
            bt[selected];
        avg_tat += tat[selected];
        avg_wct += wct[selected];
        completed++;
    }
}

```

```

    }
}

avg_tat /= n;
avg_wt /= n;

printf("\nPriority Scheduling (Pre-
Emptive):\n");
printf("PID | pPri | tAT | tBT | tCT | tTAT
| tRT | n");

for(i=0; i<n; i++) {
    printf("P%.d | %.d | %.d | %.d | %.d |
%.d | %.d | %.d | n",
    i+1, pa[i], at[i], bt[i], ct[i],
    tat[i], wt[i], rt[i]);
}

printf("\nAverage Turn Around Time :
%.6f ms\n", avg_tat);
printf("Average waiting time : %.6f ms\n",
avg_wt);

return 0;
}

```

OUTPUT:

Enter number of processes : 5.

Process 1.

Enter Process ID : 1

Enter Arrival Time : 3

Enter Burst Time : 5

Enter Priority (lower number = higher priority) : 2

Process 2

Enter Process ID : 2

Enter Arrival Time : 2

Enter Burst Time : 3

Enter Priority (lower number = higher priority) : 2

Process 3

Enter Process ID : 3

Enter Arrival Time : 3

Enter Burst Time : 5

Enter Priority (lower number = higher priority) : 2

Process 4

Enter Process ID : 4

Enter Arrival Time : 4

Enter Burst Time : 4

Enter Priority (lower number = higher priority) : 4

Process 5

Enter Process ID : 5

Enter Arrival Time : 6

Enter Burst Time : 1

Enter Priority (lower number = higher priority) : 1.

PID	AT	BT	PR	CT	TAT	WT	RT
1	3	5	2	9	6	1	0
2	2	3	2	11	9	6	0
3	3	5	2	16	13	8	8
4	4	4	4	20	16	12	12
5	6	1	1	7	1	0	0

Average Waiting Time = 5.4

Average TAT = 9

d) Round Robin (Experiment with different quantum sizes for RR algorithm)

Program:

```
#include<stdio.h>

int main() {

    int n,pid[20],at[20],bt[20],rem[20],ct[20],tat[20],wt[20],rt[20];

    int tq,time=0,done,i;

    float avgwt=0,avgtat=0;

    printf("Name: Rashmita Kharvi\n");

    printf("USN : 1BM25CS502\n");

    printf("Enter number of processes: ");

    scanf("%d",&n);

    printf("\nEnter Process ID:\n");

    for(i=0;i<n;i++) scanf("%d",&pid[i]);

    printf("Enter Arrival Time:\n");

    for(i=0;i<n;i++) scanf("%d",&at[i]);

    printf("Enter Burst Time:\n");

    for(i=0;i<n;i++){

        scanf("%d",&bt[i]);

        rem[i]=bt[i];

        rt[i]=-1;

    }

    printf("Enter Time Quantum: ");

    scanf("%d",&tq);

    do{
```

```

done=1;

for(i=0;i<n;i++){

    if(rem[i]>0 && at[i]<=time){

        if(rt[i]==-1)

            rt[i]=time-at[i];

        done=0;

        if(rem[i]>tq){

            rem[i]-=tq;

            time+=tq;

        }

        else{

            time+=rem[i];

            rem[i]=0;

            ct[i]=time;

        }

    }

}

if(done) time++;

}while(!done || time<100);

for(i=0;i<n;i++){

    tat[i]=ct[i]-at[i];

    wt[i]=tat[i]-bt[i];

    avgwt+=wt[i];

    avgtat+=tat[i];

```

```

}

printf("\nRound Robin Scheduling\n\n");

printf("PID\tAT\tBT\tCT\tTAT\tWT\tRT\n");

for(i=0;i<n;i++)

    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",

        pid[i],at[i],bt[i],ct[i],tat[i],wt[i],rt[i]);

printf("\nAverage Turnaround Time = %.2f",avgtat/n);

printf("\nAverage Waiting Time = %.2f",avgwt/n);

return 0;

}

```

Output:

```

Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of processes: 5

Enter Process ID:
1 2 3 4 5
Enter Arrival Time:
0 3 2 4 5
Enter Burst Time:
5 3 6 3 1
Enter Time Quantum: 2

Round Robin Scheduling

PID      AT      BT      CT      TAT      WT      RT
1         0       5      15      15      10       0
2         3       3      16      13      10       6
3         2       6      18      16      10       0
4         4       3      14      10       7       0
5         5       1       7       2       1       1

```

Round Robin Scheduling

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, tq, i;
```

```
    printf("Enter number of processes:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter Time Quantum:");
```

```
    scanf("%d", &tq);
```

```
    int pid[n], at[n], bt[n], rt[n];
```

```
    int wt[n], tat[n];
```

```
    int current_time = 0, completed = 0;
```

```
    for(i = 0; i < n; i++) {
```

```
        printf("\n Process %d ID:", i+1);
```

```
        scanf("%d", &pid[i]);
```

```
        printf("Arrival Time:");
```

```
        scanf("%d", &at[i]);
```

```
        printf("Burst Time:");
```

```
        scanf("%d", &bt[i]);
```

```
        at[i] = bt[i];
```

```
    }
```

```
    while (completed < n) {
```

```
        int executed = 0;
```

```
        for (i = 0; i < n; i++) {
```

```
            if (at[i] <= current_time &&
```

```
                at[i] > 0) {
```

```
                executed = 1;
```

```

if (at[i] > tq) {
    current_time += tq;
    wt[i] -= tq;
}
else {
    current_time += at[i];
    tat[i] = current_time - at[i];
    wt[i] = tat[i] - bt[i];
    at[i] = 0;
    completed++;
}
}
}
}

```

```

if (executed == 0) {
    current_time++;
}
}

```

```

float avg_wt = 0, avg_tat = 0;

```

```

printf("\nPID | TAT | BT | WT | TAT\n");
for (i = 0; i < n; i++) {

```

```

    printf("%d | %d | %d | %d | %d\n",
        pid[i], at[i], bt[i], wt[i], tat[i]);

```

```

    avg_wt += wt[i];
    avg_tat += tat[i];
}

```

```

printf

```

```
printf("\nAverage Waiting Time =
%.2f", avg-wt/n);
printf("\nAverage Turnaround Time =
%.2f", avg-tat/n);
```

```
return 0;
```

```
}
//
```

OUTPUT:

Enter Number of Processes: 5

Enter Time Quantum: 2

Process 1

Enter Process ID: 1

Enter Arrival Time: 1

Enter Burst Time: 3

Process 3

Enter Process ID: 3

Enter Arrival Time: 2

Enter Burst Time: 1

Process 4

Enter Process ID: 4

Enter Arrival Time: 3

Enter Burst Time: 3

PID	AT	BT	CT	TAT	WT	RT
1	0	5	13	13	8	0
2	1	3	12	11	8	1
3	2	1	5	3	2	2
4	3	2	9	6	4	4
5	4	3	14	10	7	5

Average Waiting Time = 5.80

Average Turnaround Time = 8.60

- 2. Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.**

Program :

```
#include<stdio.h>

int main() {
    int n,at[20],bt[20],type[20],ct[20],tat[20],wt[20];
    int i,j,time=0,temp,id[20];
    float avgwt=0,avgtat=0;
    printf("Name: Rashmita Kharvi\n");
    printf("USN : 1BM25CS502\n");
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++) {
        id[i]=i+1;
        printf("\nProcess %d\n",i+1);
        printf("Arrival Time: ");
        scanf("%d",&at[i]);
        printf("Burst Time: ");
        scanf("%d",&bt[i]);
        printf("Type (1-System, 2-User): ");
        scanf("%d",&type[i]);
    }
    for(i=0;i<n-1;i++)
        for(j=i+1;j<n;j++)
            if(type[i]>type[j] ||
                (type[i]==type[j] && at[i]>at[j])) {
```

```

        temp=id[i]; id[i]=id[j]; id[j]=temp;
        temp=at[i]; at[i]=at[j]; at[j]=temp;
        temp=bt[i]; bt[i]=bt[j]; bt[j]=temp;
        temp=type[i]; type[i]=type[j]; type[j]=temp;
    }

for(i=0;i<n;i++) {
    if(time<at[i])
        time=at[i];
    time+=bt[i];
    ct[i]=time;
    tat[i]=ct[i]-at[i];
    wt[i]=tat[i]-bt[i];
    avgwt+=tat[i];
    avgwt+=0;
    avgtat+=tat[i];
    avgwt+=wt[i];
}
avgwt=0;
for(i=0;i<n;i++)
    avgwt+=wt[i];
printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
for(i=0;i<n;i++)
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        id[i],at[i],bt[i],ct[i],tat[i],wt[i]);
printf("\nAverage Turnaround Time = %.2f",avgtat/n);
printf("\nAverage Waiting Time = %.2f",avgwt/n);
return 0;
}

```

Output

```
Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of processes: 4

Process 1
Arrival Time: 0
Burst Time: 3
Type (1-System, 2-User): 1

Process 2
Arrival Time: 0
Burst Time: 4
Type (1-System, 2-User): 2

Process 3
Arrival Time: 0
Burst Time: 3
Type (1-System, 2-User): 1

Process 4
Arrival Time: 10
Burst Time: 5
Type (1-System, 2-User): 2
```

PID	AT	BT	CT	TAT	WT
P1	0	3	3	3	0
P3	0	3	6	6	3
P2	0	4	10	10	6
P4	10	5	15	5	0

```
Average Turnaround Time = 6.00
Average Waiting Time = 2.25
```

Multi-level Queue Scheduling

```
#include <stdio.h>
int main() {
    printf("*****\n");
    int n, curr-time=0, i, j;
    int ns=0, nr=0;
    int type[10]; tq;

    int completedSys=0, completedUgr=0;
    int q[10], f=0, r=-1;
    int inq[10] = {0}, remt[10];

    int sysPID[10], sysAT[10], sysBT[10],
    sysCT[10], sys[10], sysWT[10];

    int usrPID[10], usrAT[10], usrBT[10],
    usrCT[10], usrTAT[10], usrWT[10];
    double tAT-sum=0, wt-sum=0;
    printf("Enter no. of processes:");
    scanf("%d", &n);
    //input
    for(i=0; i<n; i++) {
        printf("Enter type for process\n");
        printf("[%d] (0=System, 1=user):", i+1);
        scanf("%d", &type[i]);
        printf("Enter AT and BT for\n");
        printf("process [%d]:", i+1);
        if (type[i] == 0) {
            scanf("%d %d", &sysAT[ns],
                &sysBT[ns]);
        }
    }
}
```

```

        sysPID[ns] = i + 1;
        remt[ns] = sysBT[ns];
        ns++;
    } else {
        scanf("%d %d", &usrAT[neu], &usrBT[neu]);
        usrPID[neu] = i + 1;
        neu++;
    }
}

printf("Enter time quantum:");
scanf("%d", &tq);
while (completedSys < ns || completedUser < neu) {
    for (i = 0; i < ns; i++) {
        if (sysAT[i] <= curr_time && remt[i] > 0 && !inq[i]) {
            r = (r + 1) % 10;
            q[r] = i;
            inq[i] = 1;
        }
    }

    if (r == (r + 1) % 10) {
        int ranUser = 0;
        for (i = 0; i < neu; i++) {
            if (usrBT[i] > 0 && usrAT[i] <= curr_time) {
                curr_time += usrBT[i];
                usrCT[i] = curr_time;
                usrTAT[i] = usrCT[i] - usrAT[i];
                usrWT[i] = usrTAT[i] - usrBT[i];
                tat_sum += usrTAT[i];
                wt_sum += usrWT[i];
                usrBT[i] = 0;
            }

```

```

        completedUser++;
        ranUser = 1;
        break;
    }
}

if (!ranUser)
    curr_time++;
continue;
}

i = q[f];
f = (f+1)%10;
inq[i] = 0;
if (f == (r+1)%10) {
    f = 0;
    r = -1;
}

if (remt[i] < tq) {
    curr_time += remt[i];
    remt[i] = 0;
} else {
    curr_time += tq;
    remt[i] = tq;
}

for (j=0; j<n; j++) {
    if (SysAT[j] <= curr_time &&
        remt[j] > 0 && !inq[j]) {
        r = (r+1)%10;
        q[r] = j;
        inq[j] = 1;
    }
}

if (remt[i] != 0) {

```

```
sysCT[i] = curr-time;
```

```
sysTAT[i] = sysCT[i] - sysAT[i];
```

```
sysWT[i] = sysTAT[i] - sysBT[i];
```

```
tat-sum += sysTAT[i];
```

```
wat-sum += sysWT[i];
```

```
completedSys++;
```

```
} else {
```

```
    r = (r+1) % 10;
```

```
    q[r] = i;
```

```
    inq[i] = 1;
```

```
}
```

```
}
```

```
printf("\n PID\tAT\tBT\tType\tCT\tTAT\n\t\t\t\t\t\t\tWT\n");
```

```
for(j=1; j<=n; j++) {
```

```
    for(i=0; i<n; i++) {
```

```
        if (sysPID[i] == j) {
```

```
            printf("P%d\t%d\t%d\t%d\t%d\n",
```

```
                sysPID[i], sysAT[i], sysBT[i],
```

```
                sysCT[i], sysTAT[i], sysWT[i]);
```

```
        }
```

```
    }
```

```
for(i=0; i<n; i++) {
```

```
    if (usrPID[i] == -j) {
```

```
        printf("P%d\t%d\t%d\t%d\tUser\n",
```

```
                usrPID[i], usrAT[i], usrBT[i],
```

```
                usrCT[i], usrTAT[i], usrWT[i]);
```

```
    }
```

```
}
```

```

}
printf("\n Average TAT : %.2f\n",
      tot_sum / n);
printf(" Average WT : %.2f\n",
      wt_sum / n);
return 0;
}

```

Output :

Enter No. of Processes : 5
 Enter type for process [1] : 0
 Enter AT & BT for process [1] : 0 5
 Enter type for process [2] : 1
 Enter AT & BT for process [2] : 1 4
 Enter type of process [3] : 0
 Enter AT & BT of process [3] : 2 6
 Enter type of process [4] : 1
 Enter ~~para~~ AT & BT of process [4] : 3 3
 Enter type of process [5] : 0
 Enter AT & BT of process [5] : 5 9

PID	AT	BT	Type	CT	TAT	WT
P1	0	5	Syst	7	7	2
P2	1	0	Usr	19	18	14
P3	2	6	Syst	13	11	5
P4	3	0	Usr	22	19	16
P5	5	4	Syst	15	10	6

Average TAT : 22.00

Average WT : 10.60

3. Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

a) Rate Monotonic

Program :

```
#include<stdio.h>
#include<stdio.h>
#include<math.h>
int gcd(int a,int b){
    while(b){
        int t=a%b;
        a=b;
        b=t;
    }
    return a;
}
int lcm(int a,int b){
    return (a*b)/gcd(a,b);
}
int main(){
    int n,bt[10],p[10],i,j,L;
    float util=0,limit;
    printf("Name: Rashmita Kharvi\n");
    printf("USN : 1BM25CS502\n");
    printf("Enter the number of processes:");
    scanf("%d",&n);
    printf("Enter the CPU burst times:\n");
    for(i=0;i<n;i++)
```

```

    scanf("%d",&bt[i]);
printf("Enter the time periods:\n");
for(i=0;i<n;i++)
    scanf("%d",&p[i]);

L=p[0];
for(i=1;i<n;i++)
    L=lcm(L,p[i]);
printf("LCM=%d\n",L);
printf("\nRate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");
for(i=0;i<n;i++){
    printf("%d\t%d\t%d\n",i+1,bt[i],p[i]);
    util+=(float)bt[i]/p[i];
}
limit=n*(1.0);
for(i=0;i<n;i++)
    limit*=pow(2.0,1.0/n);
limit-=n;
printf("\n%f <= %f => %s\n",
    util,limit,
    util<=limit?"true":"false");
printf("Scheduling occurs for %d ms\n\n",L);
for(i=0;i<L;i++){
    int run=-1;
    for(j=0;j<n;j++){
        if(i%p[j]<bt[j]){
            run=j;
            break;
        }
    }
}
if(run!=-1)
    printf("%dms onwards: CPU is idle\n",i);
else

```

```

        printf("%dms onwards: Process %d running\n",i,run+1);
    }
    return 0;
}

```

Output

```

Name: Rashmita Kharvi
USN : 1BM25CS502
Enter the number of processes:2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID      Burst   Period
1         20      50
2         35     100

0.750000 <= 2.000000 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
1ms onwards: Process 1 running
2ms onwards: Process 1 running
3ms onwards: Process 1 running
4ms onwards: Process 1 running
5ms onwards: Process 1 running
6ms onwards: Process 1 running
7ms onwards: Process 1 running
8ms onwards: Process 1 running
9ms onwards: Process 1 running
10ms onwards: Process 1 running
11ms onwards: Process 1 running
12ms onwards: Process 1 running
13ms onwards: Process 1 running
14ms onwards: Process 1 running
15ms onwards: Process 1 running
16ms onwards: Process 1 running

```

Write a C program to simulate Real-Time CPU Scheduling algorithms.

- Rate monotonic
- Earliest - Deadline First
- Proportional scheduling

a) Rate monotonic scheduling (RMS)

```
#include <math.h>
```

```
#include <stdio.h>
```

```
int gcd(int a, int b) {
```

```
    if (b == 0)
```

```
        return a;
```

```
    return gcd(b, a % b);
```

```
}
```

```
int lcm(int a, int b) {
```

```
    return (a * b) / gcd(a, b);
```

```
}
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter the number of processes:");
```

```
    scanf("%d", &n);
```

```
    int burst[n], period[n];
```

```
    printf("Enter the CPU burst times:\n");
```

```
    for(int i = 0; i < n; i++) {
```

```
        scanf("%d", &burst[i]);
```

```
    }
```

```
    printf("Enter the time periods:\n");
```

```
    for(int i = 0; i < n; i++) {
```

```
        scanf("%d", &period[i]);
```

```
    }
```

```
    int hyper = period[0];
```

```
    for(int i = 1; i < n; i++) {
```

```

    hyper = lcm(hyper, period[i]);
}

printf("LCM = %d\n\n", hyper);
printf("Rate Monotone Scheduling:\n");
printf("PID\tBurst\tPeriod\n");

for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\n", i+1, burst[i],
        period[i]);
}

float u = 0;
for (int i = 0; i < n; i++) {
    u += (float) burst[i] / period[i];
}

float bound = n * (pow(2, (1/n)) - 1);
printf("\n%.f <= %.f = %.f\n", u, bound, (u <= bound) ? "true" : "false");

printf("Scheduling occurs for %d\n\n", hyper);

int remaining[n];
for (int i = 0; i < n; i++) remaining[i] = 0;
for (int t = 0; t < hyper; t++) {
    for (int i = 0; i < n; i++) {
        if (t % period[i] == 0) {
            remaining[i] = burst[i];
        }
    }
    int selected = -1;
    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            selected = i;
        }
    }
    return 0; break;
}

```

Output :

Enter the number of processes : 3

Enter the CPU Burst time : 10 25 40

Enter the time periods : 6 10 20

LCM = 60

Rate Monotone Scheduling :

PID Burst Period

1

10

6

2

25

10

3

40

20

$6.166667 \leq 0.719763 \Rightarrow \text{false}$

Schedule occurs for 60 ms

0ms onwards : Process running

1ms

2ms

3ms

4ms

5ms

6ms

b) Earliest-deadline First

Program :

```
#include<stdio.h>
int gcd(int a,int b){
    while(b){
        int t=a%b;
        a=b;
        b=t;
    }
    return a;
}
int lcm(int a,int b){
    return (a*b)/gcd(a,b);
}
int main(){
    int n,bt[10],p[10],d[10];
    int i,j,L,task;
    printf("Name: Rashmita Kharvi\n");
    printf("USN : 1BM25CS502\n");
    printf("Enter number of tasks: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("Enter burst, period, deadline for Task %d: ",i+1);
        scanf("%d%d%d",&bt[i],&p[i],&d[i]);
    }
    L=p[0];
    for(i=1;i<n;i++)
        L=lcm(L,p[i]);
    printf("\nPID\tBurst\tDeadline\tPeriod\n");
    for(i=0;i<n;i++)
        printf("%d\t%d\t%d\t%d\n",
```

```

        i+1, bt[i], d[i], p[i]);
printf("\nScheduling occurs for %d ms\n\n", L);
for(i=0; i<L; i++){
    task=-1;
    for(j=0; j<n; j++){
        if(i%p[j]<bt[j]){
            if(task==-1 || d[j]<d[task])
                task=j;
        }
    }
    if(task==-1)
        printf("%dms : CPU is idle.\n", i);

    else
        printf("%dms : Task %d is running.\n",
            i, task+1);
}
return 0;
}

```

Output

```

Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of tasks: 3
Enter burst, period, deadline for Task 1: 2 5 4
Enter burst, period, deadline for Task 2: 3 20 7
Enter burst, period, deadline for Task 3: 2 10 8

PID      Burst    Deadline    Period
1         2         4           5
2         3         7          20
3         2         8          10

Scheduling occurs for 20 ms

0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : CPU is idle.
4ms : CPU is idle.
5ms : Task 1 is running.
6ms : Task 1 is running.
7ms : CPU is idle.
8ms : CPU is idle.
9ms : CPU is idle.
10ms : Task 1 is running.
11ms : Task 1 is running.
12ms : CPU is idle.
13ms : CPU is idle.
14ms : CPU is idle.
15ms : Task 1 is running.

```

b) Earliest Deadline First : hyperion

```
#include <stdio.h>
int main() {
    int n;
    printf("Enter number of tasks :");
    scanf("%d", &n);
    int burst[n], deadline[n], period[n],
    remaining[n];
    for (int i=0; i<n; i++) {
        printf("\nTask %d:\n", i+1);
        printf("Burst Time:");
        scanf("%d", &burst[i]);
        printf("Deadline:");
        scanf("%d", &deadline[i]);
        printf("Period:");
        scanf("%d", &period[i]);
        remaining[i] = burst[i];
    }
    printf("\nPTD\tBurst\tDeadline\tPeriod\n");
    for (int i=0; i<n; i++) {
        printf("%d\t%d\t%d\t%d\n",
            i+1, burst[i], deadline[i],
            period[i]);
    }
    int time = 0, completed = 0;
    int hyper = 20;
    printf("\nScheduling occurs for %d ms\n", hyper);
    while (time < hyper) {
        int min_deadline = 9999;
        int selected = -1;
```

```

for (int i = 0; i < n; i++) {
    if (remaining[i] > 0) {
        if (deadline[i] < min_deadline) {
            min_deadline = deadline[i];
            selected = i;
        }
    }
}
if (selected == -1) {

```

```

    printf("%dms: CPU is idle.\n", time);
}
else {

```

```

    printf("%dms: Task %d is\n", time, selected + 1);
    remaining[selected]--;
}

```

```

for (int i = 0; i < n; i++) {

```

```

    if ((time + 1) % period[i] == 0) {
        remaining[i] = burst[i];
    }
}

```

```

    time++;
}

```

```

return

```

Output:

Enter number of tasks: 4

Task 1:

Burst Time: 6

Deadline: 9

Period: 15

Task 2:

Burst Time: 5

Deadline: 9

Period: 10

Task 3:

Burst Time: 10

Deadline: 14

Period: 18

Task 4:

Burst Time: 3

Deadline: 6

Period: 16

PID Burst Deadline Period

1 6 9 15

2 5 9 10

3 10 14 18

4 3 6 16

Schedule occurs for 6ms.

0ms: Task 4 is running

1ms: " 4 " "

2ms: " 4 " "

3ms: " 1 " "

4ms: " 1 " "

5ms: " 1 " "

6ms: " 1 " "

c) **Proportional scheduling**

Program :

```
#include<stdio.h>

int main() {
    int n,bt[10],w[10],cpu[10];
    int i,total=0,sum=0;
    printf("Name: Rashmita Kharvi\n");
    printf("USN : 1BM25CS502\n");
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++){
        printf("\nProcess %d\n",i+1);
        printf("Enter Burst Time: ");
        scanf("%d",&bt[i]);
        printf("Enter Weight: ");
        scanf("%d",&w[i]);
        total+=w[i];
        sum+=bt[i];
    }
    printf("\nProportional Scheduling\n\n");
    printf("PID\tBT\tWeight\tCPU Share\n");
    for(i=0;i<n;i++){
        cpu[i]=(w[i]*sum)/total;
        printf("P%d\t%d\t%d\t%d\n",
            i+1,bt[i],w[i],cpu[i]);
    }
    return 0;
}
```

Output

```
Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of processes: 3

Process 1
Enter Burst Time: 20
Enter Weight: 3

Process 2
Enter Burst Time: 15
Enter Weight: 2

Process 3
Enter Burst Time: 10
Enter Weight: 1

Proportional Scheduling
```

PID	BT	Weight	CPU Share
P1	20	3	22
P2	15	2	15
P3	10	1	7

c) Proportional Scheduling

```

#include <stdio.h>
#include <stdlib.h>
typedef struct {
    char name[5];
    int tickets;
} process;

int main() {
    int n, total_tickets = 0;
    float total_T = 0.0;

    printf("Enter the number of Processes\n");
    scanf("%d", &n);

    process p[n];
    for (int i = 0; i < n; i++) {
        printf("Process %d:\n", i+1);
        printf("Process %d:\n", i+1);
        printf("Tickets: ");
        scanf("%d", &p[i].tickets);
        total_T += p[i].tickets;
    }

    printf("Proportional Share Scheduling ---\n");

    int m;
    printf("Enter the Time Period for Scheduling: ");
    scanf("%d", &m);

```

```

int fixed_tickets[5] = {29, 55, 47, 36, 12};

for (int i = 0; i < m; i++) {
    int timing_ticket = fixed_tickets[i];
    int accumulated = 0;
    int winner_index = 0;

    for (int j = 0; j < m; j++) {
        accumulated += p[j].tickets;

        if (winning_ticket <= accumulated) {
            winner_index = j;
            break;
        }
    }

    printf("Tickets Picked: %d, \n",
           winner_index,
           winning_ticket, p[winner_index].
           name);
}

for (int i = 0; i < n; i++) {
    printf("The process: %s gets \n",
           p[i].name,
           (p[i].tickets / total_T) * 100);
}

return 0;
}

```

```

for (int i = 0; i < n; i++) {
    printf("The process is %s gets\n", i);
}
return 0;

```

Output :

Enter the number of processes : 3

Process 1 :

Tickets : 12

Process 2 :

Tickets : 25

Process 3 :

Tickets : 32

Proportional Share Scheduling --

Enter the Time Period for scheduling : 8

Tickets picked : 29 , Winner : P2

Tickets picked : 55 , Winner : P3

Tickets picked : 47 , Winner : P2

The process : P1 gets 17.39% of processor Time

The process : P2 gets 36.23% of processor Time

The process : P3 gets 46.38% of processor Time

4. Write a C program to simulate:

a) Producer-Consumer problem using semaphores

b) Dining-Philosopher's problem

a) Producer-Consumer problem using semaphores.

Program :

```
#include<stdio.h>
int mutex=1, full=0, empty=5;
int item=0;
void wait(int *s){
    (*s)--;
}
void signal(int *s){
    (*s)++;
}
void producer(){
    if(empty==0){
        printf("Buffer Full\n");
        return;
    }
    wait(&empty);
    wait(&mutex);
    item++;
    printf("Produced: Item %d\n",item);
    signal(&mutex);
    signal(&full);
}
void consumer(){
    if(full==0){
        printf("Buffer Empty\n");
        return;
    }
}
```

```

wait(&full);
wait(&mutex);

printf("Consumed: Item %d\n",item);
item--;
signal(&mutex);
signal(&empty);
}
int main(){
    int ch;
    printf("Name: Rashmita Kharvi\n");
    printf("USN : 1BM25CS502\n");
    while(1){
        printf("\n1.Producer 2.Consumer 3.Exit : ");
        scanf("%d",&ch);
        switch(ch){
            case 1:
                producer();
                break;
            case 2:
                consumer();
                break;
            case 3:
                printf("Exit");
                return 0;
            default:
                printf("Invalid Choice");
        }
    }
}

```

Output

Name: Rashmita Kharvi
USN : 1BM25CS502

1.Producer 2.Consumer 3.Exit : 1
Produced: Item 1

1.Producer 2.Consumer 3.Exit : 1
Produced: Item 2

1.Producer 2.Consumer 3.Exit : 2
Consumed: Item 2

1.Producer 2.Consumer 3.Exit : 2
Consumed: Item 1

1.Producer 2.Consumer 3.Exit : 2
Buffer Empty

1.Producer 2.Consumer 3.Exit : 3
Exit

Producer - Consumer problem using semaphores.

```
#include <stdio.h>
#include <stdlib.h>
#include <semaphore.h>
#define size 5
int buffer[size];
int in = 0, out = 0, item = 1;
sem_t empty, full, mutex;
void produce() {
    if (sem_wait(&empty) != 0) {
        printf("Buffer is full! Cannot\nProduce.\n");
        return;
    }
    sem_wait(&mutex);
    buffer[in] = item;
    printf("Producer has produced: Item\n%d\n", item);
    item++;
    in = (in + 1) % size;
    sem_post(&mutex);
    sem_post(&full);
}

void consume() {
    if (sem_wait(&full) != 0) {
        printf("Buffer is empty! Cannot\nConsume.\n");
        return;
    }
    sem_wait(&mutex);
    sem_wait(&empty);
```

```
int consumed = buffer[out];
printf("Consumer has consumed:
Item %.d\n", consumed);
out = (out + 1) % SIZE;
sem_post(&mutex);
sem_post(&empty);
}

int main() {
    int choice;
    sem_init(&empty, 0, SIZE);
    sem_init("Enter your choice:");
    scanf("%d", &choice);
    switch(choice) {
        case 1:
            produce();
            break;
        case 2:
            consume();
            break;
        case 3:
            printf("Exiting...\n");
            sem_destroy(&empty);
            sem_destroy(&full);
            sem_destroy(&mutex);
            exit(0);
        default:
            printf("Invalid choice!\n");
    }
}

return 0;
}
```

Output:

1. Producer

2. Consumer

3. Exit

Enter your choice : 1

Producer has produced : Item 1

1. Producer

2. Consumer

3. Exit

Enter your choice : 2

Producer has produced : Item 2

Enter your choice : 2

Consumer has consumed : Item 1

Enter your choice : 2

Consumer has consumed : Item 2.

b) Dining-Philosopher's problem

Program :

```
#include<stdio.h>
#include<pthread.h>
#include<semaphore.h>
#include<unistd.h>
sem_t fork1[5];
void *philosopher(void *num){
    int id=*(int*)num;
    printf("Philosopher %d is thinking.\n",id);
    sem_wait(&fork1[id]);
    printf("Philosopher %d picked left fork %d.\n",id,id);
    sem_wait(&fork1[(id+1)%5]);
    printf("Philosopher %d picked right fork %d.\n",
    id,(id+1)%5);
    printf("Philosopher %d is eating.\n",id);
    sleep(1);
    sem_post(&fork1[id]);
    sem_post(&fork1[(id+1)%5]);
    printf("Philosopher %d put down forks %d and %d.\n",
    id,id,(id+1)%5);
    printf("Philosopher %d is thinking.\n",id);
    return NULL;
}
int main(){
    pthread_t p[5];
    int id[5],i
    printf("Name: Rashmita Kharvi\n");
    printf("USN : 1BM25CS502\n");
    for(i=0;i<5;i++)
        sem_init(&fork1[i],0,1);
```

```

for(i=0;i<5;i++){
    id[i]=i;
    pthread_create(
        &p[i],
        NULL,
        philosopher,
        &id[i]
    );
}
for(i=0;i<5;i++)
    pthread_join(p[i],NULL);
return 0;
}

```

Output

```

Name: Rashmita Kharvi
USN : 1BM25CS502
Philosopher 0 is thinking.
Philosopher 0 picked left fork 0.
Philosopher 0 picked right fork 1.
Philosopher 0 is eating.
Philosopher 0 put down forks 0 and 1.
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 1 picked left fork 1.
Philosopher 1 picked right fork 2.
Philosopher 1 is eating.
Philosopher 1 put down forks 1 and 2.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 2 picked left fork 2.
Philosopher 2 picked right fork 3.
Philosopher 2 is eating.
Philosopher 2 put down forks 2 and 3.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 3 picked left fork 3.
Philosopher 3 picked right fork 4.
Philosopher 3 is eating.
Philosopher 3 put down forks 3 and 4.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 4 picked left fork 4.
Philosopher 4 picked right fork 0.
Philosopher 4 is eating.

```

Write a C program to simulate Dining - Philosopher's Problem

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define N 5
pthread_mutex_t forks[N];
pthread_t philosopher[N];
void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;
    while (1)
    {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);
        if (id % 2 == 0)
        {
            // Even -> left then right.
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked left fork %d.\n", id, left);

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked right fork %d.\n", id, right);
        }
        else
        {

```

```

// odd → right then left
pthread_mutex_lock(&forks[right]);
printf("Philosopher %d picked\n", id, right);
pthread_mutex_lock(&forks[left]);
printf("Philosopher %d picked left\n", id, left);
}

// Eating
printf("Philosopher %d is eating.\n", id);
sleep(2);
pthread_mutex_unlock(&forks[left]);
pthread_mutex_unlock(&forks[right]);
printf("Philosopher %d picked left\n", id, left);
}

return NULL;
}

int main()
{
    int i;
    int ids[N];
    for(i=0; i<N; i++)
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
    for(i=0; i<N; i++)
    {
        pthread_create(&philosophers[i],
            NULL, philosopher, ids[i]);
    }
}

```

```

    for (i=0; i<N; i++) {
        pthread_join (philosophers[i],
            NULL, philosopher, &ids[i]);
    }
    for (i=0; i<N; i++)
        pthread_mutex_destroy(&forks[i]);
    return 0;
}

```

Output :

Philosopher 0 put down fork 3 4 and
 Philosopher 0 is thinking
 Philosopher 4 is thinking
 Philosopher 2 is thinking
 Philosopher 3 is thinking

5. Write a C program to simulate:

a) Bankers' algorithm for the purpose of deadlock avoidance.

b) Deadlock Detection

a) Bankers' algorithm for the purpose of deadlock avoidance.

Program :

```
#include<stdio.h>

int main() {
    int n,m,alloc[10][10],max[10][10],need[10][10];
    int avail[10],finish[10]={0},safe[10];
    int i,j,k,count=0,found;
    printf("Name: Rashmita Kharvi\n");
    printf("USN : 1BM25CS502\n");
    printf("Enter number of processes: ");
    scanf("%d",&n);
    printf("Enter number of resources: ");
    scanf("%d",&m);
    printf("Enter Allocation Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&alloc[i][j]);
    printf("Enter Maximum Demand Matrix:\n");
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            scanf("%d",&max[i][j]);
    printf("Enter Available Resources:\n");
    for(i=0;i<m;i++)
        scanf("%d",&avail[i]);
    for(i=0;i<n;i++)
        for(j=0;j<m;j++)
            need[i][j]=max[i][j]-alloc[i][j];
```

```

while(count<n){
    found=0;
    for(i=0;i<n;i++){
        if(!finish[i]){
            for(j=0;j<m;j++){
                if(need[i][j]>avail[j])
                    break;
            }
            if(j==m){
                for(k=0;k<m;k++){
                    avail[k]+=alloc[i][k];
                }
                safe[count++]=i;
                finish[i]=1;
                found=1;
            }
        }
    }
    if(!found)
        break;
}

if(count==n){
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");
    for(i=0;i<n;i++){
        printf("P%d",safe[i]);
        if(i<n-1)
            printf(" -> ");
    }
}
else
    printf("\nSystem is NOT in safe state");
return 0;
}

```

Output

```
Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
```

Lab 5

a) Banker's algorithm for the purpose of deadlock avoidance.
Safety Algorithm.

1. Let $work$ & $finish$ be vectors of length m & n
initialize $work = Available$.

$finish[i] = false$ for $i = 0$ to $n-1$

2. Find an index i such that
a. $finish[i] = false$
b. $Need_i \leq work$.

3. $work = work + Allocation$
 $finish[i] = true$
go to step 2.

4. if $finish[i] = true$ for all i
then system is safe state

Program:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m, i, j, k;
```

```
    printf("Enter number of processes:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources:");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], need[n][m];
```

```
    int avail[m], finish[n], safeSeq[n];
```

```
    printf("Enter Allocation matrix: \n");
```

Lab 5

a) Banker's algorithm for the purpose of deadlock avoidance.
Safety Algorithm.

1. Let $work$ & $finish$ be vectors of length m & n
initialize $work = Available$.

$finish[i] = false$ for $i = 0$ to $n-1$

2. Find an index i such that
a. $finish[i] = false$
b. $Need_i \leq work$.

3. $work = work + Allocation$
 $finish[i] = true$
go to step 2.

4. if $finish[i] = true$ for all i
then system is safe state.

Program:

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m, i, j, k;
```

```
    printf("Enter number of processes:");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources:");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], max[n][m], need[n][m];
```

```
    int avail[m], finish[n], safeSeq[n];
```

```
    printf("Enter Allocation matrix:\n");
```

```

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        scanf("%d", &alloc[i][j]);
    }
}

```

```

printf("Enter maximum demand\n");

```

```

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        scanf("%d", &max[i][j]);
    }
}

```

```

printf("Enter maximum available\nresources : \n");

```

```

for (i=0; i<m; i++) {
    scanf("%d", &avail[i]);
}

```

```

for (i=0; i<n; i++) {
    for (j=0; j<m; j++) {
        need[i][j] = max[i][j] - alloc[i][j];
    }
}

```

```

for (i=0; i<n; i++) {
    finish[i] = 0;
}

```

```

int count = 0;
while (count < n) {

```

```

    int found = 0;
    for (i=0; i<n; i++) {
        if (finish[i] == 0) {

```

```

            int possible = -1;

```

```

for(j=0; j<m; j++) {
    if (need[i][j] > avail[j]) {
        possible = 0;
        break;
    }
}
if (possible) {
    for (k=0; k<m; k++) {
        avail[k] += alloc[i][k];
    }
    safeseg[count++] = i;
    finish[i] = 1;
    found = 1;
}
}
if (found == 0) {
    break;
}
if (count == n) {
    printf("\nSystem is in a safe state.\n");
}
return 0;
}

```

Output

```

Enter number of process : 5
Enter number of resources : 3
Enter Allocation Matrix :
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

```

Enter Maximum Demand Matrix: $\begin{bmatrix} 7 & 5 & 3 \\ 3 & 2 & 2 \\ 9 & 0 & 2 \\ 2 & 2 & 2 \\ 4 & 3 & 3 \end{bmatrix}$

Enter available Resources: $3 \ 3 \ 2$

System is in a safe state

Safe sequence: $P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

Example Trace:

1. $work = 3 \ 3 \ 2$

$Finish[i] = \text{false} \quad i = 0 \text{ to } 4.$

2. $i = 0$ $Finish[0] = \text{false}$

$Need_0 \leq work.$

$7 \ 4 \ 3 \leq 3 \ 3 \ 2 \rightarrow \text{false}$

$i = 1$ $Finish[1] = \text{false}$

$Need_1 \leq work$

$1 \ 2 \ 2 \leq 3 \ 3 \ 2 \rightarrow \text{True}$

3. $work = work + Allocation.$

$= 3 \ 3 \ 2$

$+ 2 \ 0 \ 0$

$5 \ 3 \ 2$

1. $Finish[2] = \text{false}$

$Need_2 \leq work.$

$6 \ 0 \ 0 \leq 5 \ 3 \ 2 \rightarrow \text{false}$

$Finish[3] = \text{false}$

$Need_3 \leq work$

$0 \ 1 \ 1 \leq 5 \ 3 \ 2 \rightarrow \text{True}$

$work = work + allocation$

$$= \begin{array}{r} 532 \\ + 211 \\ \hline 743 \end{array}$$

$$+ 211$$

$$743$$

Finish[4] = false

Need₄ ≤ work

4 3 1 ≤ 7 4 3 → True

work = work + allocation

$$= \begin{array}{r} 743 \\ + 002 \\ \hline 745 \end{array}$$

$$745$$

Finish[0] = false

Need₀ ≤ work

7 4 3 ≤ 7 4 5 → True

$$w = w + A = \begin{array}{r} 745 \\ + 010 \\ \hline 755 \end{array}$$

$$010$$

$$755$$

Finish[2] = false

Need₂ ≤ work

6 0 0 ≤ 7 5 5 → True

$$w = w + A = \begin{array}{r} 755 \\ + 302 \\ \hline 1057 \end{array}$$

$$302$$

$$1057$$

Safe sequence : P₁ → P₃ → P₄ → P₀ → P₂

b) Deadlock Detection

Program :

```
#include<stdio.h>

int main() {
    int n,m,alloc[10][10],req[10][10];
    int avail[10],finish[10]={0},safe[10];
    int i,j,k,count=0,found;
    printf("Name: Rashmita Kharvi\n");
    printf("USN : 1BM25CS502\n");
    printf("Enter the number of processes: ");
    scanf("%d",&n);
    printf("Enter the number of resources: ");
    scanf("%d",&m);
    printf("Enter the allocation matrix:\n");
    for(i=0;i<n;i++){
        printf("Process %d: ",i);
        for(j=0;j<m;j++)
            scanf("%d",&alloc[i][j]);
    }
    printf("Enter the request matrix:\n");
    for(i=0;i<n;i++){
        printf("Process %d: ",i);
        for(j=0;j<m;j++)
            scanf("%d",&req[i][j]);
    }
    printf("Enter the available resources: ");
    for(i=0;i<m;i++)
        scanf("%d",&avail[i]);

    while(count<n){
        found=0;
        for(i=0;i<n;i++){
```

```

        if(!finish[i]){
            for(j=0;j<m;j++)
                if(req[i][j]>avail[j])
                    break;
            if(j==m){
                for(k=0;k<m;k++)
                    avail[k]+=alloc[i][k];
                safe[count++]=i;
                finish[i]=1;
                found=1;
            }
        }
    }
    if(!found)
        break;
}
if(count==n){
    printf("\nSystem is in safe state.\n");
    printf("Safe Sequence is: ");
    for(i=0;i<n;i++)
        printf("P%d ",safe[i]);
}
else{
    printf("\nDeadlock detected.\n");
    printf("Processes in deadlock: ");
    for(i=0;i<n;i++)
        if(!finish[i])
            printf("P%d ",i);
}
return 0;
}

```

Output

```
Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
F:\Downloads\OS>gcc deadlock.c -o deadlock

F:\Downloads\OS>.\deadlock
Name: Rashmita Kharvi
USN : 1BM25CS502
Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0

System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1
```

b. Deadlock Detection

Write (several instance Bankers Algorithm) algorithm, program and output.

Algorithm

1. work and finish be vectors of length m
 - a. work = Available
 - b. for $i \leftarrow 1$ to n
 - if Allocation _{i} $\neq 0$ then
Finish $[i]$ = false
2. Find an index i such that
 - a. Finish $[i]$ = false
 - b. Request _{i} $\leq$ work
 if no such i exists go to step 4
3. work = work + available
Finish $[i]$ = True.
go to step 2.
4. if finish $[i]$ = false
for some i
 $1 \leq i \leq n$
the system is in deadlock state
P _{i} is in deadlock.

Program

```
#include <stdio.h>

int main() {
    int n, m, i, j, k;
    printf("Enter the number of processes:");
    scanf("%d", &n);
    printf("Enter the number of resources:");
    scanf("%d", &m);
    int allocation[n][m], request[n][m];
    int available[m];
    int finish[n], safeSequences[n];
    printf("Enter the allocation matrix:\n");
    for (i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (j = 0; j < m; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("Enter the request matrix:\n");
    for (i = 0; i < n; i++) {
        printf("Process %d: ", i);
        for (j = 0; j < m; j++) {
            scanf("%d", &request[i][j]);
        }
    }

    printf("Enter the available resources:");
    for (i = 0; i < m; i++) {
        scanf("%d", &available[i]);
    }

    for (i = 0; i < n; i++) {
        int flag = 0;
        for (j = 0; j < m; j++) {
```

```

if (allocation[i][j] != 0) {
    flag = 1;
    break;
}
}
if (flag == 1)
    finish[i] = 0;
else
    finish[i] = 1;
}
int count = 0;
while (count < n) {
    int found = 0;
    for (i = 0; i < n; i++) {
        if (finish[i] == 0) {
            int possible = 1;
            for (j = 0; j < m; j++) {
                if (request[i][j] > available[j])
                    possible = 0;
                break;
            }
            if (possible) {
                for (k = 0; k < m; k++) {
                    available[k] += allocation[i][k];
                }
                safeSequence[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }
}
}

```

```

    if (found == 0)
        break;
    }
    int deadlock = 0;
    for (i = 0; i < n; i++) {
        if (finish[i] == 0) {
            deadlock = 1;
            break;
        }
    }
    if (deadlock == 0) {
        printf("System is in safe state.\n");
        printf("Safe Sequence is : ");
        for (i = 0; i < count; i++) {
            printf("P%d", safeSequence[i]);
        }
    }
    else {
        printf("Deadlock detected in the\nsystem.\n");
        printf("Processes involved in deadlock\nare : ");
        for (i = 0; i < n; i++) {
            if (finish[i] == 0) {
                printf("P%d", i);
            }
        }
    }
    return 0;
}

```

Output :

Enter the number of processes : 5

Enter the number of resources : 3

Enter the allocation matrix :

Process 0 : 0 1 0

Process 1 : 2 0 0

Process 2 : 3 0 3

Process 3 : 2 1 1

Process 4 : 0 0 2

Enter the request matrix :

Process 0 : 0 0 0

Process 1 : 2 0 2

Process 2 : 0 0 0

Process 3 : 1 0 0

Process 4 : 0 0 2

Enter the available resources : 0 0 0

System is in safe state

Safe sequence : $P_0 \rightarrow P_2 \rightarrow P_3 \rightarrow P_4 \rightarrow P_1$

$$1. \text{work} = 000$$

$$F[i] = \text{false} \text{ to } n = 5$$

$$2. F[0] = \text{false}$$

$$R_0 \leftarrow W$$

$$(000) \leq (000) \rightarrow \text{True}$$

$$W = W + A$$

$$= 000$$

$$010$$

$$010$$

$$F[0] = \text{True}$$

$$F[i] = \text{false}$$

$$R_1 \leftarrow W$$

$$(2 \ 0 \ 2) \leq (0 \ 1 \ 0) \text{ --- false}$$

$$F[0] = F$$

$$R_0 \leq w$$

$$(0 \ 0 \ 0) \leq (0 \ 1 \ 0) \text{ --- True}$$

$$w = w + A$$

$$= 0 \ 1 \ 0$$

$$3 \ 0 \ 3$$

$$3 \ 1 \ 3$$

$$F[2] = \text{True}$$

$$F[3] = \text{false.}$$

$$R_3 \leq w$$

$$(1 \ 0 \ 0) \leq (3 \ 1 \ 3) \text{ --- True.}$$

$$w = w + A$$

$$= 3 \ 1 \ 3$$

$$2 \ 1 \ 1$$

$$5 \ 2 \ 4$$

$$F[3] = \text{True.}$$

$$F[4] = \text{True}$$

$$R_4 \leq w$$

$$(0 \ 0 \ 2) \leq (5 \ 2 \ 4) \text{ --- True.}$$

$$w = 5 \ 2 \ 4$$

$$0 \ 0 \ 2$$

$$5 \ 2 \ 6$$

$$F[4] = \text{True.}$$

$$F[0] = F$$

$$R_0 \leq w$$

$$(0 \ 0 \ 0) \leq (5 \ 2 \ 6) \text{ --- True}$$

$$w = 5 \ 2 \ 6$$

$$0 \ 1 \ 0$$

$$5 \ 3 \ 6 \text{ --- True}$$

Sequence: P_0, P_2, P_3, P_4, P_1

6. Write a C program to simulate the following contiguous memory allocation techniques.

a) Worst-fit

b) Best-fit

c) First-fit

Program :

```
#include<stdio.h>

void firstFit(int block[],int m,int process[],int n){

    int temp[20],alloc[20],i,j;

    for(i=0;i<m;i++) temp[i]=block[i];

    printf("\n--- First Fit ---\n\n");

    printf("Process No.\tProcess Size\tBlock No.\n");

    for(i=0;i<n;i++){

        alloc[i]=-1;

        for(j=0;j<m;j++){

            if(temp[j]>=process[i]){

                alloc[i]=j;

                temp[j]-=process[i];

                break;

            }

        }

        if(alloc[i]!=-1)

            printf("%d\t\t%d\t\t%d\n",

                i+1,process[i],alloc[i]+1);
```

```

        else

            printf("%d\t%d\tNot Allocated\n",

                i+1,process[i]);

        }
    }

void bestFit(int block[],int m,int process[],int n){

    int temp[20],alloc[20],i,j,best;

    for(i=0;i<m;i++) temp[i]=block[i];

    printf("\n--- Best Fit ---\n\n");

    printf("Process No.\tProcess Size\tBlock No.\n");

    for(i=0;i<n;i++){

        best=-1;

        for(j=0;j<m;j++){

            if(temp[j]>=process[i]){

                if(best==-1||temp[j]<temp[best])

                    best=j;

            }

        }

        alloc[i]=best;

        if(best!=-1)

            temp[best]-=process[i];

        if(alloc[i]!=-1)

            printf("%d\t%d\t%d\n",

                i+1,process[i],alloc[i]+1);
    }
}

```

```

        else

            printf("%d\t%d\t\tNot Allocated\n",

                i+1,process[i]);

        }
    }

void worstFit(int block[],int m,int process[],int n){

    int temp[20],alloc[20],i,j,worst;

    for(i=0;i<m;i++) temp[i]=block[i];

    printf("\n--- Worst Fit ---\n\n");

    printf("Process No.\tProcess Size\tBlock No.\n");

    for(i=0;i<n;i++){

        worst=-1;

        for(j=0;j<m;j++){

            if(temp[j]>=process[i]){

                if(worst==-1||temp[j]>temp[worst])

                    worst=j;

            }

        }

        alloc[i]=worst;

        if(worst!=-1)

            temp[worst]-=process[i];

        if(alloc[i]!=-1)

            printf("%d\t%d\t\t%d\n",

```

```

        i+1,process[i],alloc[i]+1);

    else

        printf("%d\t%d\t\tNot Allocated\n",

            i+1,process[i]);

    }

}

int main(){

    int m,n,block[20],process[20],i;

    printf("Name: Rashmita Kharvi\n");

    printf("USN : 1BM25CS502\n");

    printf("Enter number of memory blocks: ");

    scanf("%d",&m);

    printf("Enter sizes of %d memory blocks:\n",m);

    for(i=0;i<m;i++)

        scanf("%d",&block[i]);

    printf("\nEnter number of processes: ");

    scanf("%d",&n);

    printf("Enter sizes of %d processes:\n",n);

    for(i=0;i<n;i++)

        scanf("%d",&process[i]);

    firstFit(block,m,process,n);

    bestFit(block,m,process,n);

    worstFit(block,m,process,n);

    return 0;

```

}

Output

```
Name: Rashmita Kharvi
USN : 1BM25CS502
Enter number of memory blocks: 5
Enter sizes of 5 memory blocks:
100 500 200 300 600

Enter number of processes: 4
Enter sizes of 4 processes:
212 417 112 426

--- First Fit ---

Process No.      Process Size      Block No.
1                212              2
2                417              5
3                112              2
4                426             Not Allocated

--- Best Fit ---

Process No.      Process Size      Block No.
1                212              4
2                417              2
3                112              3
4                426              5

--- Worst Fit ---

Process No.      Process Size      Block No.
1                212              5
2                417              2
3                112              5
4                426             Not Allocated
```

Write a C program to simulate the following contiguous memory allocation techniques

a) Worst Fit

b) Best - Fit

c) First - Fit

```
#include <stdio.h>
```

```
void firstFit(int blockSize[], int m,
int processSize[], int n)
{
```

```
    int allocation[n];
```

```
    for (int i = 0; i < n; i++)
```

```
        allocation[i] = -1;
```

```
    for (int i = 0; i < n; i++)
```

```
        for (int j = 0; j < m; j++)
```

```
            if (blockSize[j] >= processSize[i])
```

```
            {
```

```
                allocation[i] = j;
```

```
                blockSize[j] -= processSize[i];
```

```
                break;
```

```
            }
```

```
    printf("\n First Fit Allocation:\n");
```

```
    printf("\n Process No | Process Size | Block No\n");
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        printf("%d | %d | ", i+1,
```

```
        processSize[i]);
```

```
        if (allocation[i] != -1)
```

```
            printf("%d\n", allocation[i]+1);
```

```
        else
```

```
            printf("Not Allocated");
```

```

    }
}

void bestfit (int blockSize[], int m, int
processSize[], int n)
{
    int allocation[n];
    for (int i=0; i<n; i++)
        allocation[i] = -1;
    for (int i=0; i<n; i++)
    {
        int bestIdx = -1;
        for (int j=0; j<m; j++)
            if (blockSize[j] >= processSize[i])
                if (bestIdx == -1 || blockSize[j] <
                    blockSize[bestIdx])
                    bestIdx = j;
        if (bestIdx != -1)
        {
            allocation[i] = bestIdx;
            blockSize[bestIdx] -= processSize[i];
        }
    }
}

printf("\nBestFit Allocation:\n");
printf("Process No | Process Size | Block No\n");
for (int i=0; i<n; i++)
{
    printf("%.d | %.d | %.d", i+1, processSize[i],
        allocation[i]);
    if (allocation[i] != -1)
        printf("%.d\n", allocation[i]);
    else
        printf("Not Allocated\n");
}
}

```

int main()

{

int m, n;

printf("Enter no. of memory blocks: ");

scanf("%d", &m);

int blockSize[m];

printf("Enter sizes of memory blocks:\n");

for (int i=0; i<m; i++)

scanf("%d", &blockSize[i]);

printf("Enter number of processes: ");

scanf("%d", &n);

int processSize[n];

printf("Enter sizes of processes:\n");

for (int i=0; i<n; i++)

scanf("%d", &processSize[i]);

int block1[m], block2[m], block3[m];

for (int i=0; i<m; i++)

block1[i] = block2[i] = block3[i] = blockSize[i];

FirstFit(block1, m, processSize, n);

bestfit(block2, m, processSize, n);

worstfit(block3, m, processSize, n);

return 0;

}

Output:

Enter no. of memory blocks: 5

Enter size of 5 memory blocks:

100 500 200 300 600

Enter number of processes: 4

Enter sizes of 4 processes:

212 417 912 426

- First Fit -

Process No.	Process Size	Block No.
-------------	--------------	-----------

1	212	2
---	-----	---

2	417	5
---	-----	---

3	112	2
4	426	Not allocated
Best fit		
1	212	4
2	417	2
3	112	3
4	426	5
Worst fit		
1	212	5
2	417	2
3	112	5
4	426	Not allocated

7. Write a C program to simulate page replacement algorithms.

a) FIFO

b) LRU

c) Optimal

Program :

```
#include<stdio.h>

void print(int f[],int n){

    int i;

    printf("[");

    for(i=0;i<n;i++){

        if(f[i]==-1)

            printf(" ");

        else

            printf("%d",f[i]);

        if(i<n-1)

            printf(" ");

    }

    printf("]\n");

}

void fifo(int p[],int n,int fn){

    int f[10],i,j=0,k,hit,pf=0;

    for(i=0;i<fn;i++) f[i]=-1;

    printf("\n--- FIFO Page Replacement ---\n");

    for(i=0;i<n;i++){

        hit=0;
```

```

    for(k=0;k<fn;k++)

        if(f[k]==p[i])

            hit=1;

    if(!hit){

        f[j]=p[i];

        j=(j+1)%fn;

        pf++;

    }

    printf("Page %d -> ",p[i]);

    print(f,fn);

}

printf("Total Page Faults (FIFO): %d\n",pf);
}

void lru(int p[],int n,int fn){

    int f[10],time[10]={0};

    int i,j,k,min,hit,pf=0;

    for(i=0;i<fn;i++) f[i]=-1;

    printf("\n--- LRU Page Replacement ---\n");

    for(i=0;i<n;i++){

        hit=0;

        for(j=0;j<fn;j++){

            if(f[j]==p[i]){

                hit=1;

                time[j]=i;

```

```

        }

    }

    if(!hit){
        min=0;

        for(j=1;j<fn;j++)

            if(time[j]<time[min])

                min=j;

        f[min]=p[i];

        time[min]=i;

        pf++;

    }

    printf("Page %d -> ",p[i]);

    print(f,fn);

}

printf("Total Page Faults (LRU): %d\n",pf);
}

void optimal(int p[],int n,int fn){

    int f[10],i,j,k,pos,far,hit,pf=0;

    for(i=0;i<fn;i++) f[i]=-1;

    printf("\n--- Optimal Page Replacement ---\n");

    for(i=0;i<n;i++){

        hit=0;

        for(j=0;j<fn;j++)

```

```

        if(f[j]==p[i])

            hit=1;
if(!hit){
    pos=-1;

    far=i;

    for(j=0;j<fn;j++){

        for(k=i+1;k<n;k++){

            if(f[j]==p[k])

                break;

            if(k>far){

                far=k;

                pos=j;

            }

        }

    }

    if(pos== -1)

        for(j=0;j<fn;j++){

            if(f[j]== -1){

                pos=j;

                break;

            }

        }

    if(pos== -1)

        pos=0;

    f[pos]=p[i];

```

```

        pf++;

    }

    printf("Page %d -> ",p[i]);

    print(f,fn);

}

printf("Total Page Faults (Optimal): %d\n",pf);

}

int main(){

    int n,p[30],fn,i;

    printf("Name: Rashmita Kharvi\n");

    printf("USN : 1BM25CS502\n");

    printf("Enter number of pages: ");

    scanf("%d",&n);

    printf("Enter page reference string:\n");

    for(i=0;i<n;i++)

        scanf("%d",&p[i]);

    printf("Enter number of frames: ");

    scanf("%d",&fn);


    fifo(p,n,fn);

    lru(p,n,fn);

    optimal(p,n,fn);

    return 0;

}

```

Output

```
Name: Rashmita Kharvi
USN : IBM25CS582
Enter number of pages: 12
Enter page reference string:
7 0 1 2 0 3 0 4 2 3 0 3
Enter number of frames: 3

---- FIFO Page Replacement ----
Page 7 -> [7      ]
Page 0 -> [7 0     ]
Page 1 -> [7 0 1   ]
Page 2 -> [2 0 1   ]
Page 0 -> [2 0 1   ]
Page 3 -> [2 3 1   ]
Page 0 -> [2 3 0   ]
Page 4 -> [4 3 0   ]
Page 2 -> [4 2 0   ]
Page 3 -> [4 2 3   ]
Page 0 -> [0 2 3   ]
Page 3 -> [0 2 3   ]
Total Page Faults (FIFO): 10

---- LRU Page Replacement ----
Page 7 -> [7      ]
Page 0 -> [0      ]
Page 1 -> [0 1     ]
Page 2 -> [0 1 2   ]
Page 0 -> [0 1 2   ]
Page 3 -> [0 3 2   ]
Page 0 -> [0 3 2   ]
Page 4 -> [0 3 4   ]
Page 2 -> [0 2 4   ]
Page 3 -> [3 2 4   ]
Page 0 -> [3 2 0   ]
Page 3 -> [3 2 0   ]
Total Page Faults (LRU): 9

---- Optimal Page Replacement ----
Page 7 -> [7      ]
Page 0 -> [0      ]
Page 1 -> [0 1     ]
Page 2 -> [0 2     ]
Page 0 -> [0 2     ]
Page 3 -> [0 2 3   ]
Page 0 -> [0 2 3   ]
Page 4 -> [4 2 3   ]
Page 2 -> [4 2 3   ]
Page 3 -> [4 2 3   ]
Page 0 -> [0 2 3   ]
Page 3 -> [0 2 3   ]
Total Page Faults (Optimal): 7
```

Lab 7.

Write a C program to simulate replacement algorithms

a) FIFO

b) LRU

c) Optimal

```
#include <stdio.h>
```

```
void fifo(int pages[], int n, int frames, int count) {
```

```
    int frames[10], i, j, k=0, fault=0, found;
```

```
    for (i=0; i<frames-count; i++)
```

```
        frames[i] = -1;
```

```
    printf("\n - Fifo Page Replacement\n");
```

```
    for (i=0; i<n; i++) {
```

```
        found = 0;
```

```
        for (j=0; j<frames-count; j++) {
```

```
            if (frames[j] == pages[i]) {
```

```
                found = 1;
```

```
                break;
```

```
            }
```

```
        }
```